# Data Structures and Algorithms I.




University of Szeged

Robert Nagy

2025/26. Autumn Semester

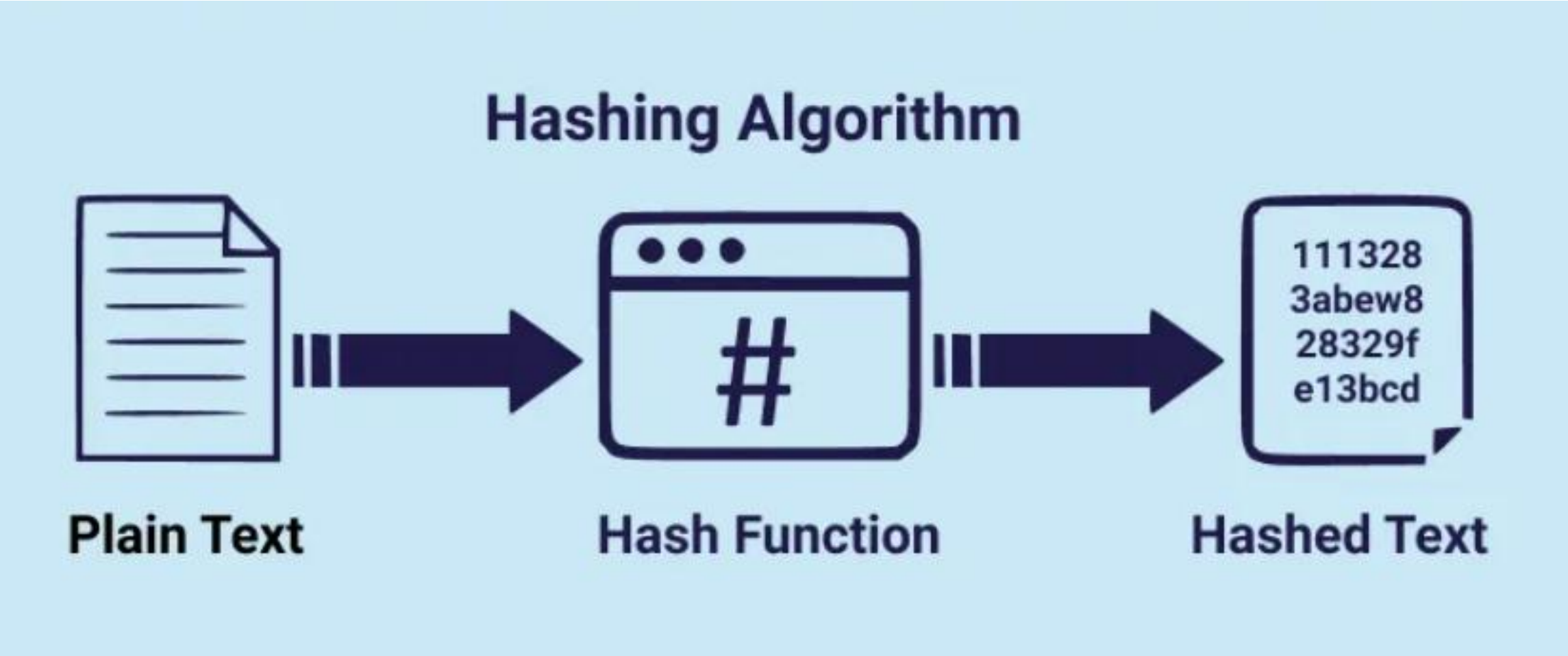
# Hash tables



# Hash function

Transform the input

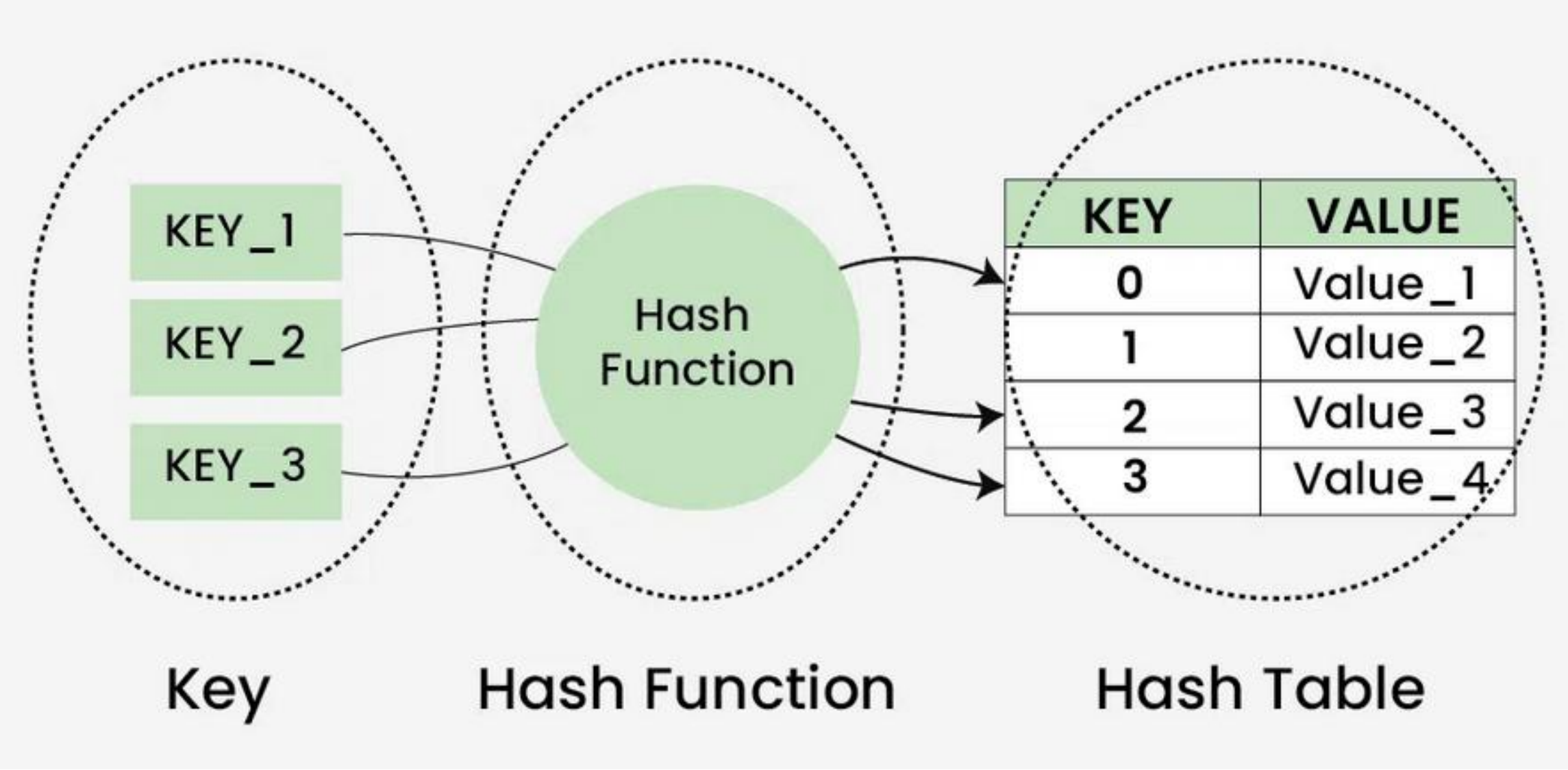
https://medium.com/nybles/hashing-algorithms-d10171ca2e89

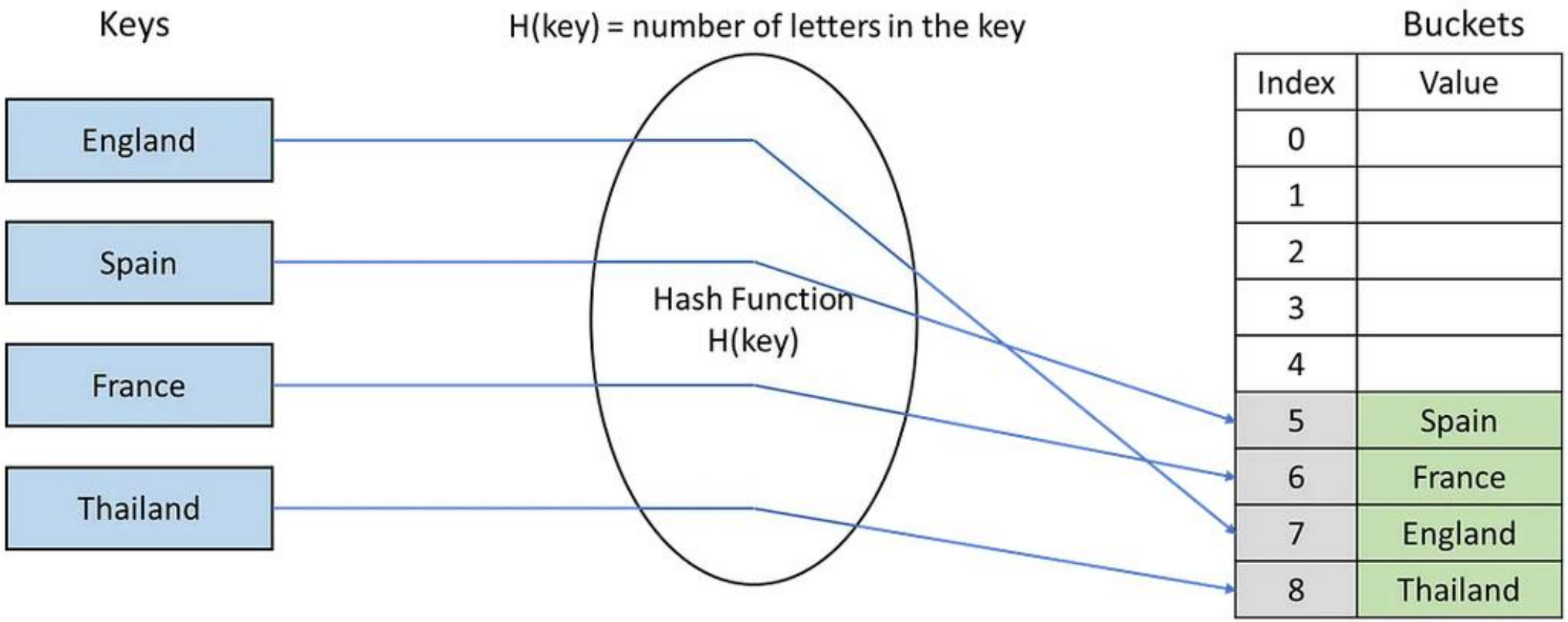


# Popular hashing algorithms

| |
|---|
| SHA-256 |
| SHA-3 |
| CRC32 |
| **MD5** |

https://www.geeksforgeeks.org/dsa/hash-table-data-structure/




Keys
H(key) = number of letters in the key
Buckets
England
Spain
France
Thailand
Hash Function
H(key)
Index
Value
0
1
2
3
4
5
Spain
6
France
7
England
8
Thailand

# Motivation

- Storage for unique elements, with ***O(1)* search time**.
  - Unordered set
  - Unordered map

$$\text{Load factor} = \frac{k}{n}$$

- The size of the table → $n$
- The number of occupied slots → $k$

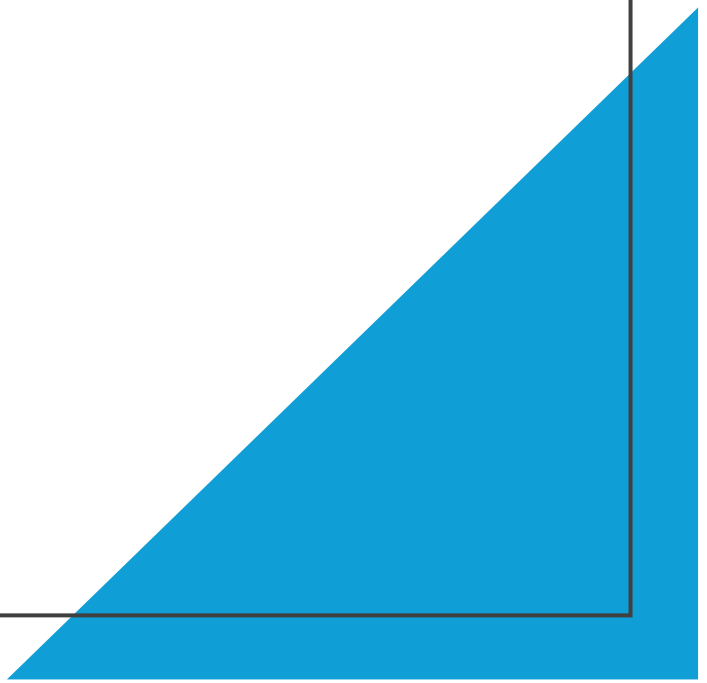
# Rehashing

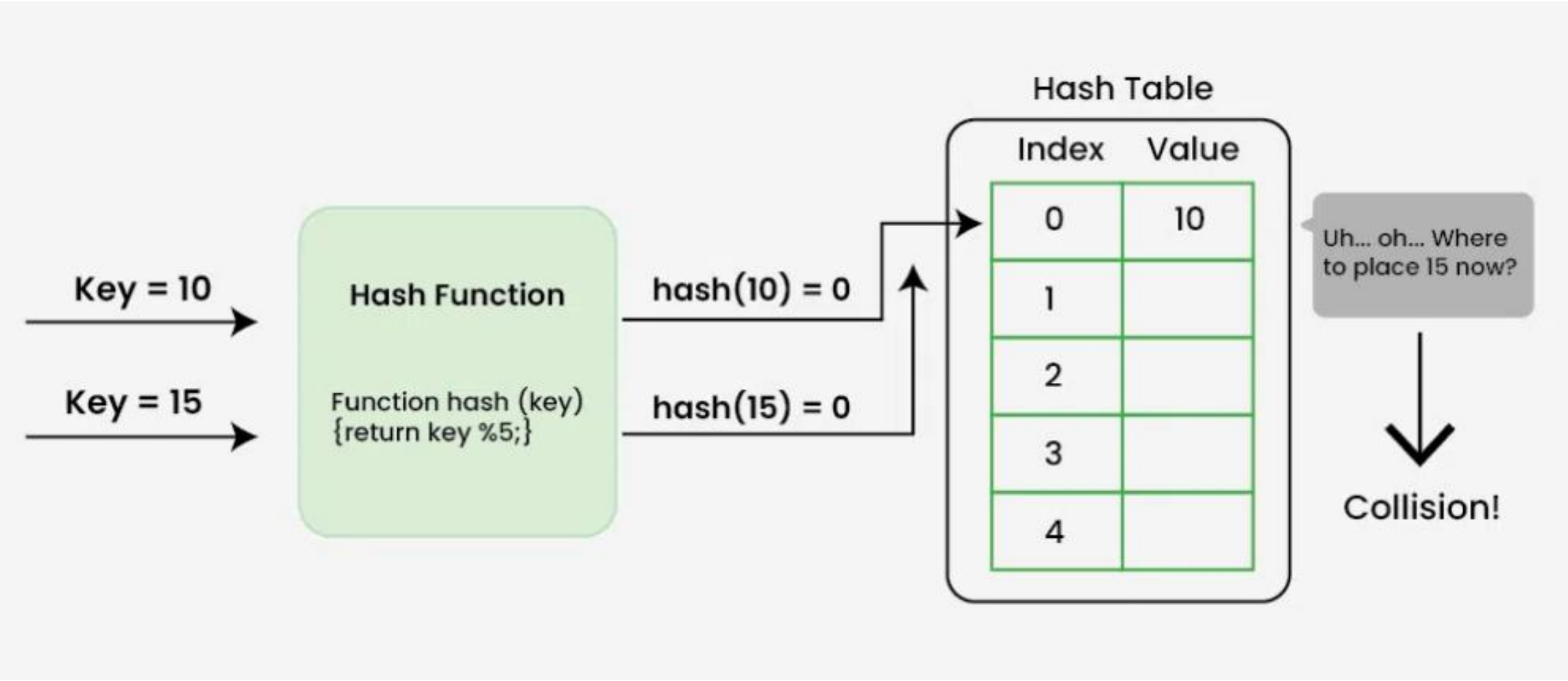
- $if\ loadFactor \geq threshold$

# Rehashing

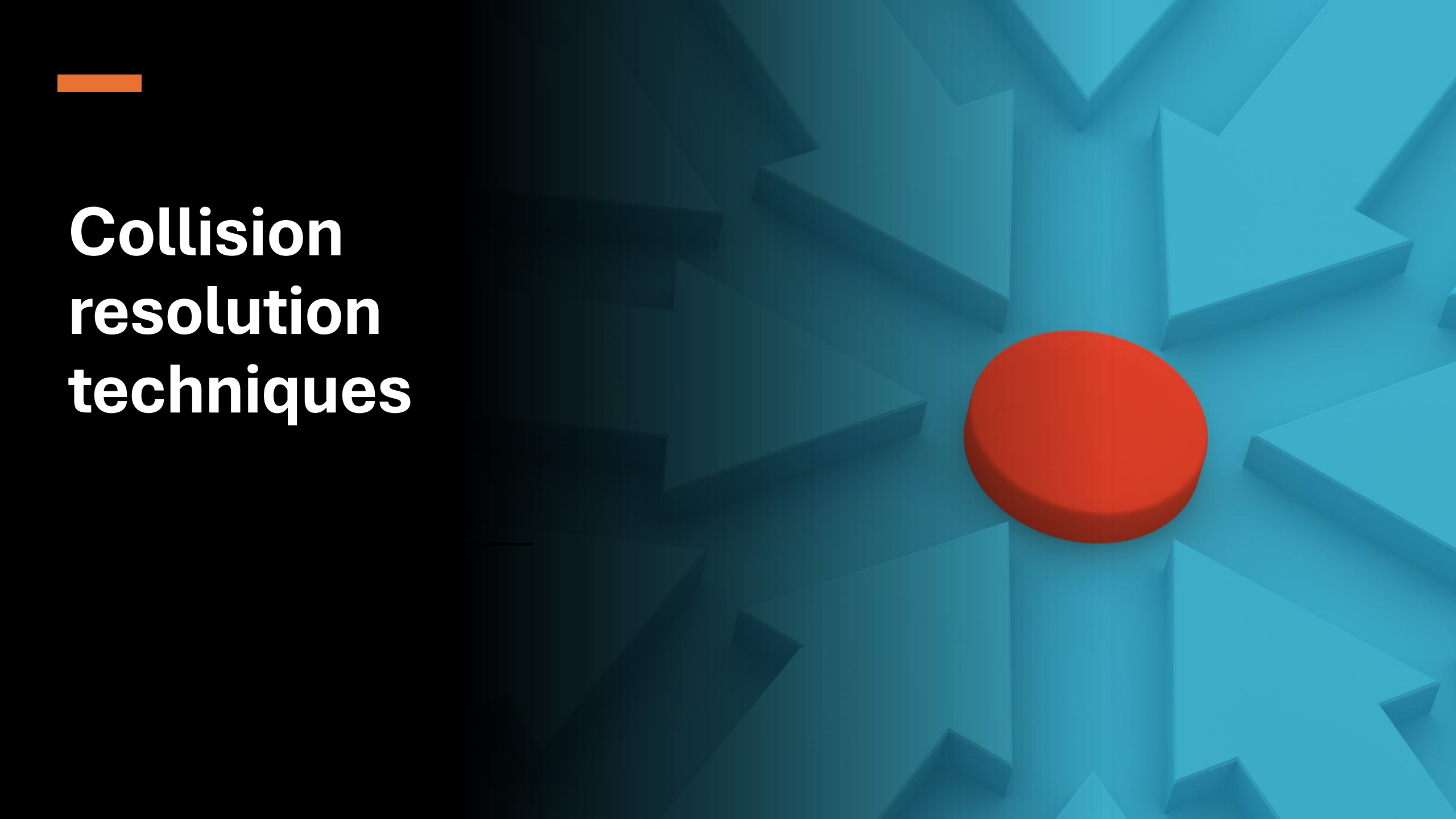
- $if\ loadFactor \geq threshold$
- Increase the table size & recalculate hashes for elements

# Collisions

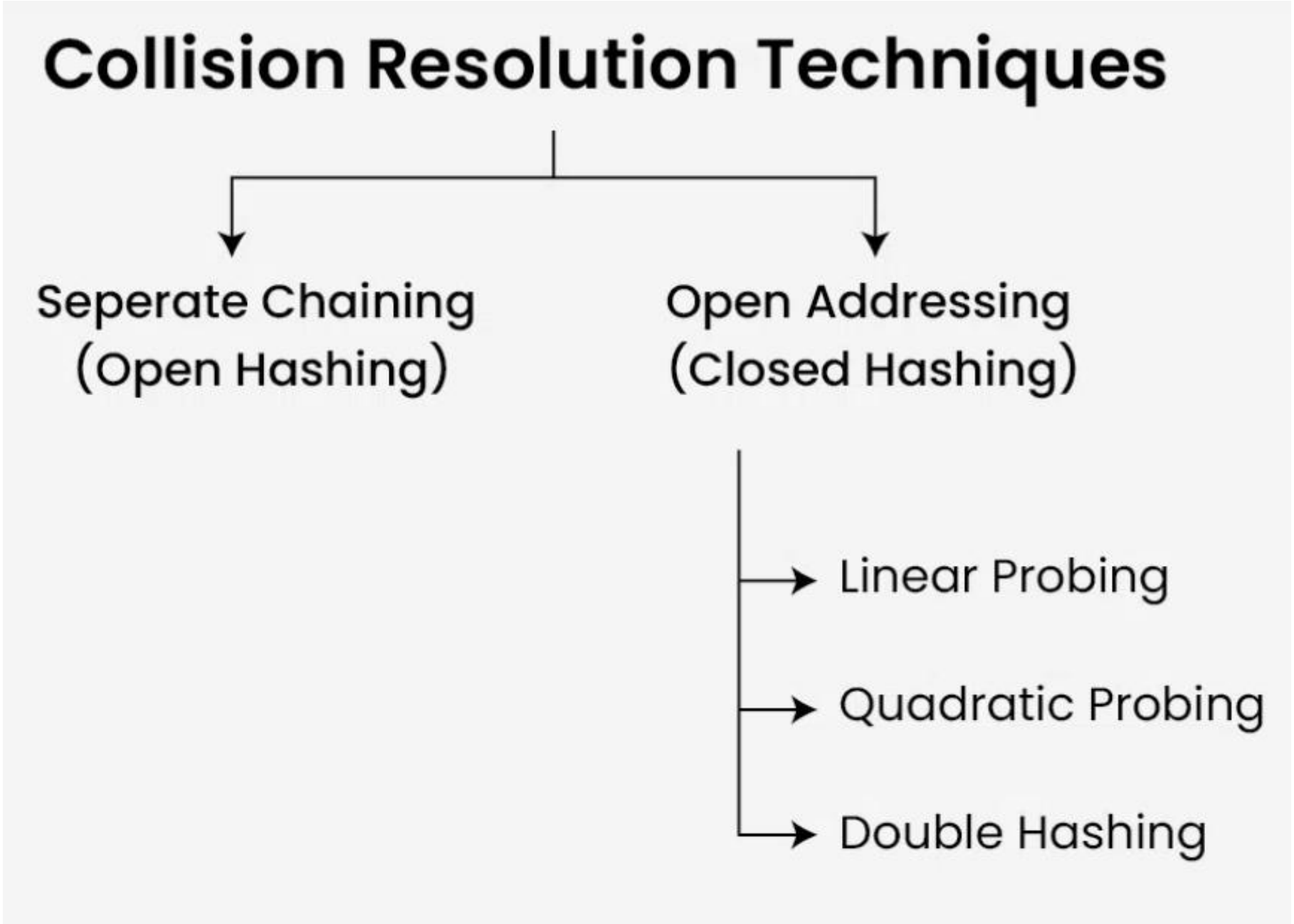


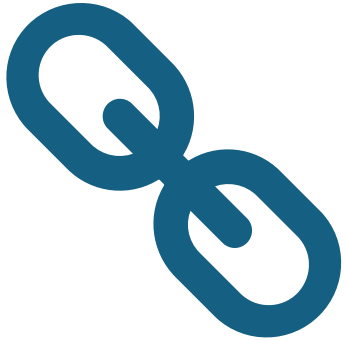

Key = 10
Key = 15
Hash Function
Function hash (key)
{return key %5;}
hash(10) = 0
hash(15) = 0
Hash Table
Index
Value
0
10
1
2
3
4
Uh... oh... Where to place 15 now?
Collision!

# Collision resolution techniques



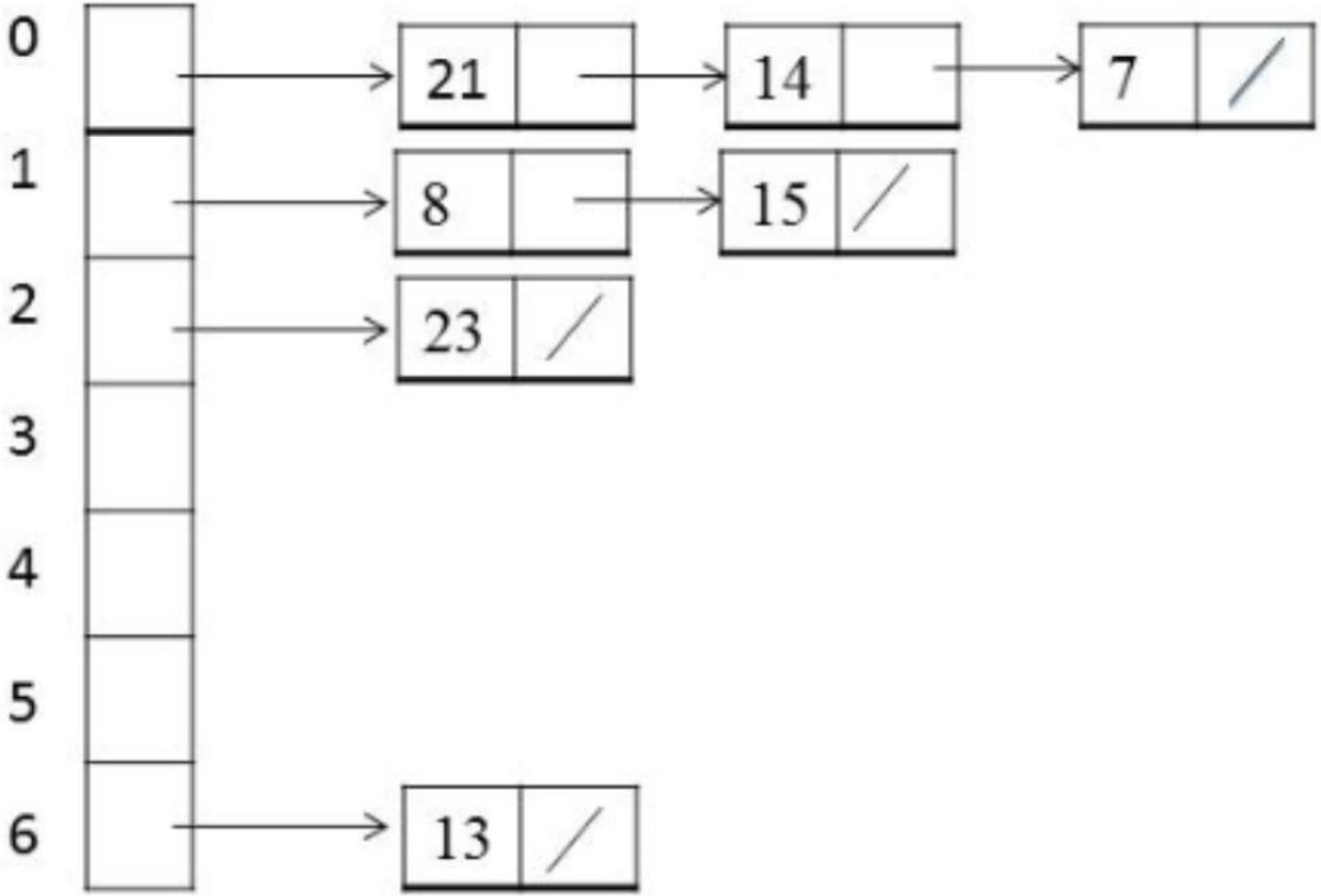
https://www.geeksforgeeks.org/dsa/collision-resolution-techniques/





# Separate chaining

https://www.researchgate.net/figure/Hash-table-representation-for-separate-chaining_fig2_284454241

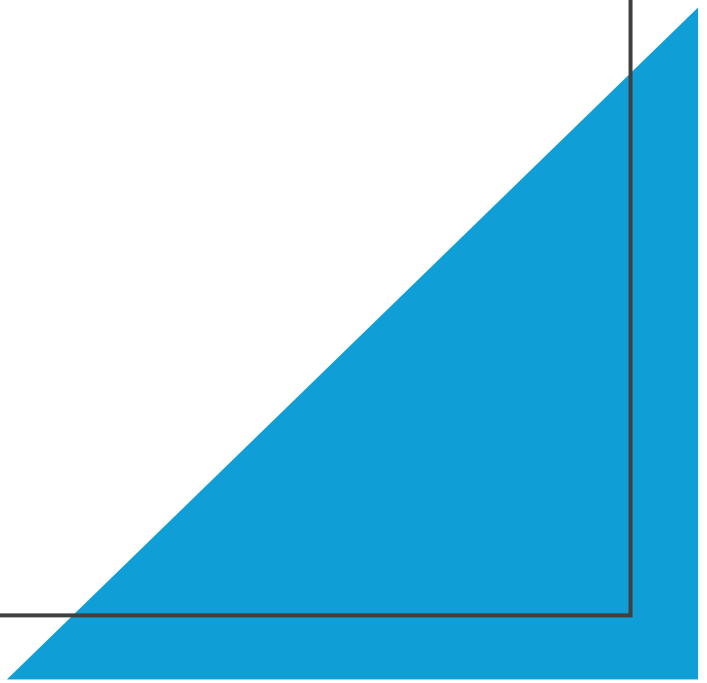


# Exercise – separate chaining

- $Hash(x) = x \ mod \ 5$
- Data stream: 4, 71, 33, 25, 99, 38, 13, 34, 1, 77
- Hash table size = 5

$$Hash(key) = (key + i)\ mod\ n$$

# Linear probing



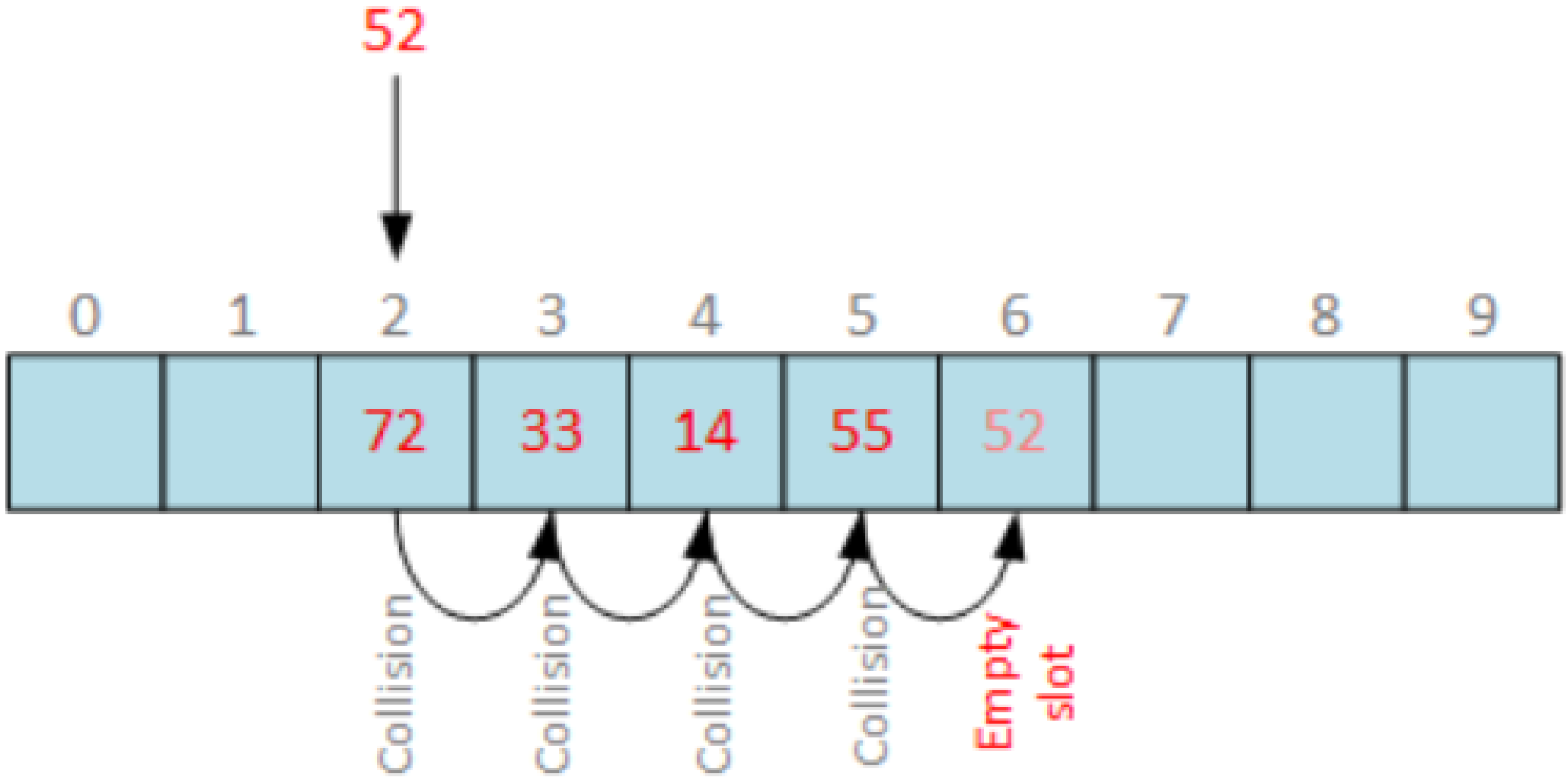
$$Hash(key) = (key + i)\ mod\ n$$

$$i = 0, 1, 2, \ldots$$

$$n = len(table)$$

# Linear probing

https://www.sysadmins.lv/blog-en/array-search-hash-tables-behind-the-scenes.aspx



# Problem – Primary Clustering

$$h(k,i) = h(k) + c_1 i + c_2 i^2 \pmod{m}$$

Quadratic probing

$$H + 1^2 , H + 2^2 , H + 3^2 , H + 4^2 \ldots\ldots\ldots\ldots\ldots\ldots H + k^2$$

# Example

# Exercise

- $Hash(x) = (x + i^2) \, mod \, table_size$
  - $i = 0, 1, 2, \ldots$
- Data: 7, 14, 21, 23, 5, 6
- *Table size = 7*

# F.Y.I.

- Abseil – Google's C++ util lib – Hash uses Quadratic probing
  - https://github.com/abseil/abseil-cpp

- Open problem:

It is conjectured[3][11] that quadratic probing, when filled to $1 - \epsilon$ full, supports insertions in expected time $O(\epsilon^{-1})$. Proving this, or even proving *any* non-trivial time bound for quadratic probing remains open.[11]

# Double hashing

$$h(i,k) = (h_1(k) + i \cdot h_2(k)) \bmod |T|$$

# Example

- $hash(i, x) = \big(f(x) + ig(x)\big)\, mod\ table_size$
- $f(x) = 2x + 3$
- $g(x) = 6 + x(x + 1)$
- Data: 3, 2, 1, 4, 7
- Table size = 5